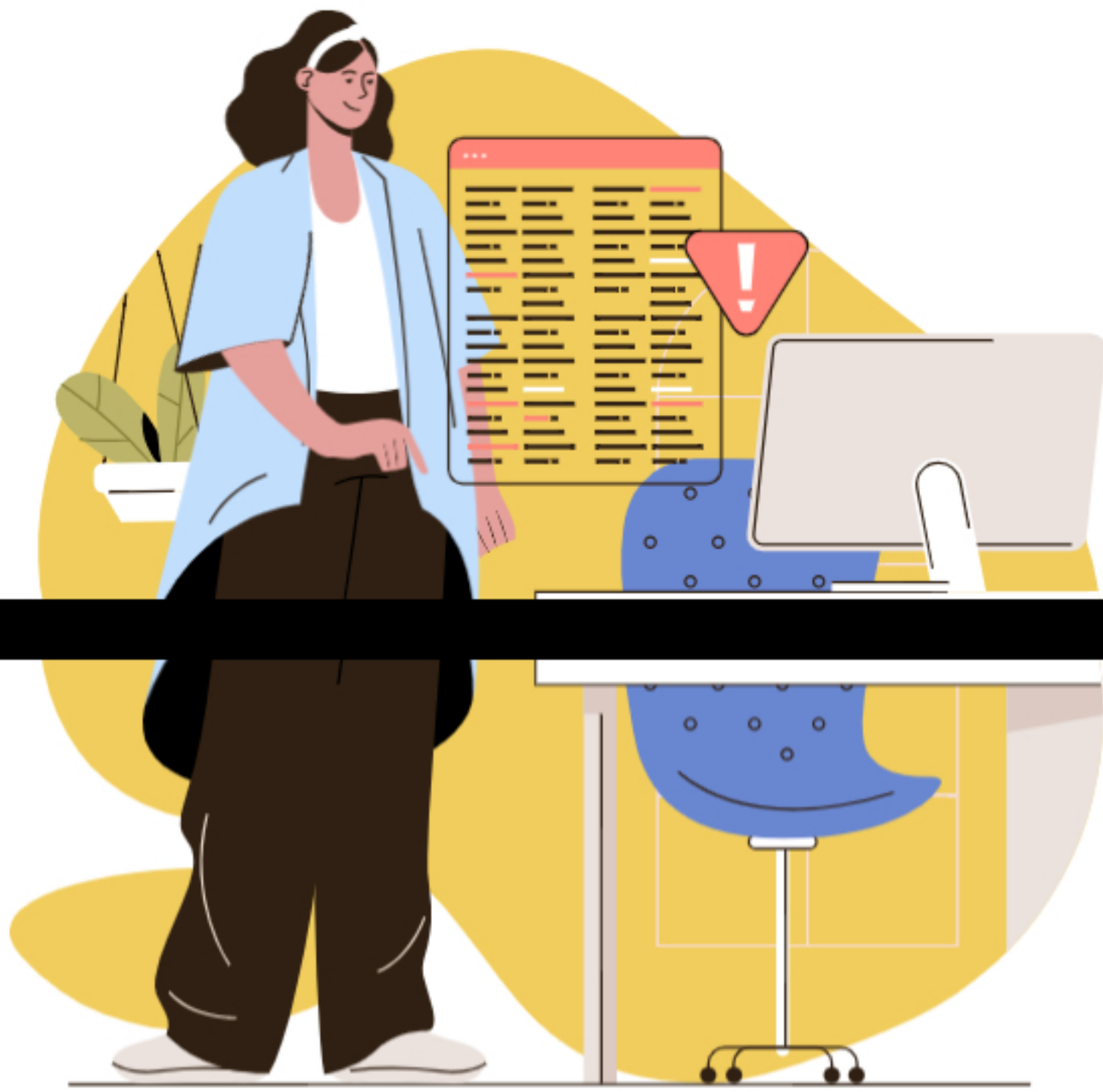


Versionamento com DBT

O versionamento com DBT (Data Build Tool) é uma prática essencial para garantir que as transformações de dados e os modelos sejam desenvolvidos, mantidos e implantados de forma organizada e controlada.

Vejamos a seguir os principais aspectos e práticas recomendadas para o versionamento com DBT.



Integração com Git

DBT é frequentemente integrado com sistemas de controle de versão como Git, o que permite um gerenciamento eficiente do código e das mudanças. Git fornece uma maneira robusta de rastrear alterações, colaborar com outros desenvolvedores e manter um histórico claro de versões.

O Git é um sistema de controle de versão distribuído que permite a desenvolvedores rastrear alterações em arquivos, colaborar em projetos e gerenciar versões de código-fonte de forma eficiente. Ele facilita o trabalho em equipe, permitindo que múltiplas pessoas trabalhem no mesmo projeto ao mesmo tempo, mantendo um histórico detalhado de todas as mudanças feitas ao longo do tempo. O Git é amplamente utilizado em desenvolvimento de software e é a base para plataformas de colaboração como GitHub e GitLab.

1. Repositório Git:

O código DBT, incluindo modelos, macros, seeds, snapshots e documentação, é armazenado em um repositório Git. Isso permite que todas as alterações sejam versionadas, garantindo que você possa reverter para versões anteriores quando necessário.

2. Branches (Ramos):

A criação de branches é uma prática comum no desenvolvimento com DBT. Um branch principal (geralmente chamado de main ou master) contém a versão estável do projeto, enquanto branches de desenvolvimento ou feature branches são utilizados para desenvolver novas funcionalidades ou fazer ajustes. Após o desenvolvimento e os testes, as alterações são mescladas de volta ao branch principal.

3. Pull Requests (Requisições de Pull):

Utilizar pull requests permite revisões de código e facilita a colaboração. Antes de mesclar um branch de desenvolvimento ao branch principal, uma pull request é criada, permitindo que outros membros da equipe revisem e aprovelem as mudanças. Isso ajuda a garantir a qualidade e a consistência do código.

Estrutura de Pastas e Arquivos

Manter uma estrutura de pastas e arquivos bem organizada é essencial para o versionamento eficaz em DBT. Isso facilita a navegação e a gestão de mudanças no projeto.

1. Modelos:

Organize os modelos em diretórios lógicos baseados em esquemas ou funcionalidades. Por exemplo, você pode ter pastas separadas para staging, intermediate, e marts.

2. Macros e Seeds:

Armazene macros em um diretório macros, enquanto seeds podem ser armazenados em um diretório data. Isso ajuda a manter a clareza e a modularidade do projeto.

3. Snapshots e Documentação:

Snapshots podem ser organizados em um diretório **snapshots**, e a documentação pode ser mantida em docs. Essa separação ajuda na manutenção e na aplicação de controles de versão adequados.

Controle de Versão de Artefatos de Dados

Além do código, é importante versionar artefatos de dados gerados pelas transformações do DBT.

1. Versionamento de Tabelas:

Em alguns casos, pode ser necessário manter versões históricas de tabelas de saída. Isso pode ser feito criando tabelas com nomes versionados ou utilizando snapshots do DBT para capturar o estado dos dados ao longo do tempo.

2. Monitoramento e Logs:

Manter logs de execução e monitoramento também é importante. Ferramentas de CI/CD (Integração Contínua/Entrega Contínua) podem ser integradas para automatizar execuções e registrar logs, facilitando o rastreamento de mudanças e a resolução de problemas.

Boas Práticas

Seguir algumas boas práticas pode melhorar ainda mais o processo de versionamento com DBT.

1. Commits Atômicos:

Faça commits pequenos e atômicos, que representem uma única mudança lógica. Isso facilita o rastreamento e a revisão das alterações.

2. Mensagens de Commit Claras:

Escreva mensagens de commit claras e descritivas, explicando o que foi alterado e por quê. Isso ajuda na compreensão do histórico de mudanças.

4. Testes Automatizados:

Utilize testes automatizados para validar as transformações e garantir que as mudanças não introduzam erros. DBT permite a criação de testes de dados que podem ser executados automaticamente como parte do pipeline de CI/CD.

4. Documentação Atualizada:

Mantenha a documentação do projeto atualizada com as mudanças de versão. Isso inclui descrições de modelos, métricas e outras transformações de dados.

O versionamento com DBT, quando bem implementado, permite uma gestão eficaz das transformações de dados, promovendo colaboração, rastreabilidade e qualidade no desenvolvimento de projetos de dados.

